CRIMINAL EYE

AI Model Download, Local Setup, Wiring & Next.js Integration

A reproducible engineering guide for your Windows laptop:
Ryzen 7 7435HS • 24 GB RAM • RTX 4050 Laptop GPU • 6 GB VRAM

Locked recommendation
Qwen3-8B Q4_K_M → structured facial attributes → geometry/component layer → Stable Diffusion 1.5 + ControlNet Lineart → forensic-style sketch.

What you will build

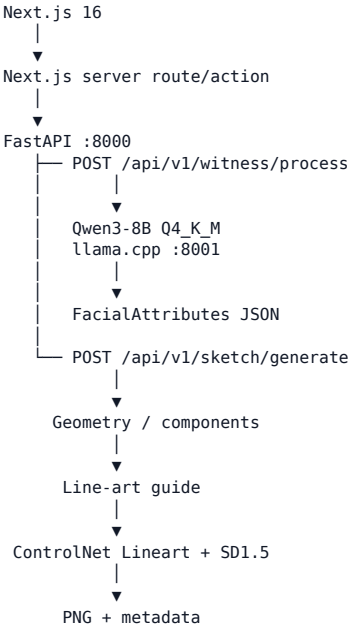
- Local Qwen witness-description parser.
- Strict facial-attribute JSON with a controlled vocabulary.
- Deterministic eyes/nose/mouth/face geometry.
- SD1.5 + ControlNet Lineart rendering.
- MediaPipe landmark support.
- FS2K research dataset organization.
- FastAPI orchestration service.
- Next.js server-side integration.
- 6 GB VRAM-safe execution and testing.

Hardware rule
Do not load Qwen3-8B BF16 or a 27B-class LLM on this GPU. Use Q4_K_M with llama.cpp and keep LLM/diffusion workloads sequential.

Sources checked: official model/runtime documentation, 4 September 2026.

1. Architecture & Responsibilities

Do not use a single prompt-to-image call. The system needs controlled placement of facial features, so language understanding, geometry and rendering are separate layers.



Layer	Responsibility
Next.js	UI, authentication, case workflow, review/edit UX
FastAPI	Validation, orchestration, stable API contract
Qwen3-8B	Witness text → observable facial attributes
Geometry	Attribute → normalized feature placement / line art
SD1.5 + ControlNet	Structural guide → final sketch rendering

Important boundary
Qwen should not identify a person or determine guilt. The generated sketch is a reconstruction aid and should remain human-reviewed.

2. Official Download Links

Use the official repositories below. The exact Qwen Q4_K_M file is the recommended local quantization for your 6 GB GPU.

Qwen3-8B GGUF — official
https://huggingface.co/Qwen/Qwen3-8B-GGUF

Qwen3-8B Q4_K_M — exact file
https://huggingface.co/Qwen/Qwen3-8B-GGUF/blob/main/Qwen3-8B-Q4_K_M.gguf

llama.cpp — official releases
https://github.com/ggml-org/llama.cpp/releases

Stable Diffusion 1.5 — official
https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5

ControlNet Lineart — official
https://huggingface.co/lllyasviel/control_v11p_sd15_lineart

FS2K — official GitHub
https://github.com/DengPingFan/FS2K

MediaPipe Face Landmarker — Python
https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker/python

Hugging Face CLI
https://huggingface.co/docs/huggingface_hub/guides/cli

Diffusers installation
https://huggingface.co/docs/diffusers/installation

Diffusers ControlNet
https://huggingface.co/docs/diffusers/main/en/using-diffusers/controlnet

PyTorch — local install
https://pytorch.org/get-started/locally/

FastAPI — first steps
https://fastapi.tiangolo.com/tutorial/first-steps/

Do not download
Do not use Qwen3-8B BF16, Qwen3.6-27B, FLUX-class large checkpoints or large training pipelines as your primary local setup.

3. Windows + Python Environment

Keep the Next.js project and AI service independent. Your frontend remains pnpm-based; the AI service gets its own Python virtual environment.

3.1 Verify system

```
python --version
git --version
nvidia-smi
node --version
pnpm --version
```

3.2 Create ai-service

```
# From the Criminal Eye repository root
mkdir ai-service
cd ai-service

py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
```

3.3 Create model/data folders

```
mkdir models
mkdir models\qwen
mkdir models\sd15
mkdir models\controlnet
mkdir models\mediapipe
mkdir datasets
mkdir datasets\FS2K
mkdir outputs
mkdir tests
```

3.4 Python dependencies

```
# requirements.txt
fastapi
uvicorn[standard]
pydantic
pydantic-settings
httpx
python-multipart
pillow
opencv-python
numpy
mediapipe
transformers
accelerate
diffusers[torch]
safetensors
huggingface_hub

python -m pip install -r requirements.txt
```

PyTorch
Use the official PyTorch selector for the current Windows + NVIDIA CUDA wheel. Verify the current command at installation time rather than copying an old CUDA command.

4. Qwen3-8B Q4_K_M + llama.cpp

Qwen is the witness-description parser. It should output constrained JSON, not the final image.

4.1 Install llama.cpp

```
winget install llama.cpp
llama --help
```

4.2 Start Qwen

```
llama serve -hf Qwen/Qwen3-8B-GGUF:Q4_K_M --host 127.0.0.1 --port 8001
```

4.3 Test the local OpenAI-compatible API

```
$body = @{
  model = "Qwen3-8B-GGUF"
  messages = @(
    @{
      role = "user"
      content = "Return only JSON: {"face_shape":"oval"}"
    }
  )
  temperature = 0.1
} | ConvertTo-Json -Depth 8

Invoke-RestMethod `
  -Uri "http://127.0.0.1:8001/v1/chat/completions" `
  -Method Post `
  -ContentType "application/json" `
  -Body $body
```

4.4 Production prompt contract

```
SYSTEM:
Extract only observable facial characteristics from the witness description.
Return JSON matching the supplied schema.
Do not identify a person.
Do not invent missing attributes.
Use "unknown" when the description is insufficient.

USER:
<witness description>

OUTPUT:
{
  "face_shape": "...",
  "eyes": {...},
  "eyebrows": {...},
  "nose": {...},
  "mouth": {...},
  "jaw": {...},
  "chin": {...}
}
```

Keep Qwen private
Bind it to 127.0.0.1. The browser must never call port 8001 directly.

5. Facial Attributes → Geometry

The geometry layer is the control mechanism for your eyes/nose/mouth/face-shape datasets. It should convert semantic attributes into normalized placement rules before diffusion.

5.1 Example attribute object

```
{
  "face_shape": "oval",
  "eyes": {
    "shape": "almond",
    "size": "medium",
    "spacing": "normal",
    "tilt": "neutral"
  },
  "eyebrows": {"thickness": "medium", "shape": "arched"},
  "nose": {
    "bridge": "straight",
    "length": "long",
    "width": "narrow",
    "tip": "rounded"
  },
  "mouth": {
    "width": "medium",
    "upper_lip": "thin",
    "lower_lip": "medium"
  },
  "jaw": {"width": "medium", "shape": "rounded"},
  "chin": {"size": "medium", "shape": "rounded"}
}
```

Region	Controlled vocabulary examples
Face	oval, round, square, oblong, heart
Eyes	almond, round, narrow; small/medium/large; close/normal/wide
Eyebrows	thin/medium/thick; straight/arched
Nose	straight/convex/concave; narrow/medium/wide; short/medium/long
Mouth	thin/medium/full; narrow/medium/wide
Jaw	narrow/medium/wide; angular/rounded
Chin	small/medium/large; pointed/rounded/square

5.2 Normalized geometry

```
{
  "canvas": {"width": 512, "height": 512},
  "anchors": {
    "left_eye": [0.36, 0.40],
    "right_eye": [0.64, 0.40],
    "nose_tip": [0.50, 0.58],
    "mouth": [0.50, 0.70],
    "chin": [0.50, 0.86]
  }
}
```

Quality test
Change only nose.width. The eyes, mouth, jaw and face proportions should remain stable as far as the conditioning pipeline allows.

6. Stable Diffusion 1.5 + ControlNet

Use Diffusers-format repositories. ControlNet provides extra structural conditioning so the renderer follows your generated line-art guide.

6.1 Download with Hugging Face CLI

```
hf --help
hf auth login   # only if authentication is required

hf download stable-diffusion-v1-5/stable-diffusion-v1-5 `
  --local-dir .\models\sd15

hf download lllyasviel/control_v11p_sd15_lineart `
  --local-dir .\models\controlnet\lineart
```

6.2 6 GB starting configuration

```
import torch
from diffusers import ControlNetModel, StableDiffusionControlNetPipeline

controlnet = ControlNetModel.from_pretrained(
    "./models/controlnet/lineart",
    torch_dtype=torch.float16,
)

pipe = StableDiffusionControlNetPipeline.from_pretrained(
    "./models/sd15",
    controlnet=controlnet,
    torch_dtype=torch.float16,
    safety_checker=None,
)

pipe.enable_model_cpu_offload()
pipe.enable_attention_slicing()

# Start: 512x512, batch=1, 20-30 steps
```

6.3 First test order

First make SD1.5 generate without ControlNet. Then add ControlNet. Finally feed your own geometry-generated line-art guide. This isolates errors.

> Compatibility
> Use the SD1.5 ControlNet Lineart checkpoint with the SD1.5 base model. Do not mix an SDXL ControlNet with this pipeline.

7. MediaPipe + FS2K

MediaPipe supplies facial landmarks/geometry; FS2K supplies paired face/sketch research data. They solve different problems.

7.1 MediaPipe

```
python -m pip install mediapipe

# Store the compatible task model here:
models/
└── mediapipe/
    └── face_landmarker.task
```

Use the official Face Landmarker guide to obtain the compatible task model. Its Python task outputs 3D landmarks and related facial information.

7.2 FS2K

The official FS2K repository describes 2,104 image-sketch pairs and provides current download instructions for photo, sketch and annotation data.

```
datasets/
└── FS2K/
    ├── photo/
    │   ├── photo1/
    │   ├── photo2/
    │   └── photo3/
    ├── sketch/
    │   ├── sketch1/
    │   ├── sketch2/
    │   └── sketch3/
    ├── anno_train.json
    ├── anno_test.json
    └── README.pdf
```

FS2K official repository
https://github.com/DengPingFan/FS2K

7.3 Your component layer

```
datasets/
├── components/
│   ├── eyes/
│   ├── eyebrows/
│   ├── noses/
│   ├── mouths/
│   ├── jaws/
│   └── face_shapes/
└── metadata/
    ├── dataset_manifest.json
    └── splits.json
```

Dataset discipline
Do not assume FS2K contains every component label you need. Build and document your own taxonomy. Keep train/validation/test splits deterministic.

8. FastAPI AI Service

FastAPI is the only backend boundary Next.js needs to understand. It owns validation, orchestration and model lifecycle.

```
ai-service/
├── app/
│   ├── main.py
│   ├── api/v1/
│   │   ├── health.py
│   │   ├── witness.py
│   │   └── sketch.py
│   ├── core/
│   │   ├── config.py
│   │   └── logging.py
│   ├── schemas/
│   │   ├── witness.py
│   │   └── sketch.py
│   ├── services/
│   │   ├── witness_service.py
│   │   ├── geometry_service.py
│   │   └── sketch_service.py
│   └── providers/
│       ├── qwen_provider.py
│       ├── geometry_provider.py
│       └── diffusion_provider.py
├── models/
├── datasets/
├── outputs/
└── tests/
```

8.1 Environment

```
AI_HOST=127.0.0.1
AI_PORT=8000
QWEN_BASE_URL=http://127.0.0.1:8001/v1
QWEN_MODEL=Qwen3-8B-GGUF
SD_MODEL_PATH=./models/sd15
CONTROLNET_MODEL_PATH=./models/controlnet/lineart
MEDIAPIPE_MODEL_PATH=./models/mediapipe/face_landmarker.task
OUTPUT_DIR=./outputs
NEXTJS_ORIGIN=http://localhost:3000
```

8.2 Endpoints

Method / endpoint	Purpose
GET /api/v1/health	Process alive
GET /api/v1/health/ready	Dependencies/models ready
POST /api/v1/witness/process	Witness text → validated attributes
POST /api/v1/sketch/generate	Attributes → geometry → sketch

8.3 Minimal server

```
from fastapi import FastAPI

app = FastAPI(
    title="Criminal Eye AI Service",
    version="0.1.0",
)

@app.get("/api/v1/health")
def health():
    return {"status": "ok", "service": "criminal-eye-ai"}
```

8.4 Run it

```
uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

9. Provider Wiring

Keep providers replaceable. The API contract should not change if you later replace Qwen or the diffusion model.

```
import httpx

class QwenProvider:
    def __init__(self, base_url: str, model: str):
        self.base_url = base_url.rstrip("/")
        self.model = model

    async def complete(self, system_prompt: str, user_text: str) -> str:
        payload = {
            "model": self.model,
            "temperature": 0.1,
            "messages": [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_text},
            ],
        }
        async with httpx.AsyncClient(timeout=90) as client:
            r = await client.post(
                f"{self.base_url}/chat/completions",
                json=payload,
            )
            r.raise_for_status()
            return r.json()["choices"][0]["message"]["content"]
```

9.1 Witness contract

```
POST /api/v1/witness/process

{
  "text": "Long oval face, narrow almond eyes, long straight nose..."
}

200
{
  "attributes": {
    "face_shape": "oval",
    "eyes": {...},
    "nose": {...},
    "mouth": {...},
    "eyebrows": {...},
    "jaw": {...},
    "chin": {...}
  },
  "warnings": []
}
```

9.2 Sketch contract

```
POST /api/v1/sketch/generate

{
  "attributes": { ... },
  "seed": 184729,
  "resolution": 512,
  "steps": 24,
  "control_strength": 0.85
}

200
{
  "job_id": "uuid",
  "status": "completed",
  "image_url": "/api/v1/sketches/uuid/image"
}
```

10. Next.js Integration

Never expose model ports to the browser. Next.js proxies requests server-side to FastAPI.

10.1 Next.js environment

```
# .env.local
AI_SERVICE_URL=http://127.0.0.1:8000
```

10.2 Route Handler

```
// app/api/ai/witness/route.ts
import { NextResponse } from "next/server";

export async function POST(request: Request) {
  const body = await request.json();

  const upstream = await fetch(
    `${process.env.AI_SERVICE_URL}/api/v1/witness/process`,
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
      cache: "no-store",
    }
  );

  return NextResponse.json(
    await upstream.json(),
    { status: upstream.status }
  );
}
```

10.3 Client call

```
const res = await fetch("/api/ai/witness", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ text: witnessDescription }),
});

if (!res.ok) throw new Error("AI processing failed");

const data = await res.json();
```

UX recommendation
Witness description → extracted attributes → investigator review/edit → generate sketch → preview → save to case.

11. Run the Complete Local System

Use three PowerShell terminals during development.

Terminal 1 — Qwen

```
cd <CRIMINAL_EYE>\ai-service
llama serve -hf Qwen/Qwen3-8B-GGUF:Q4_K_M --host 127.0.0.1 --port 8001
```

Terminal 2 — FastAPI

```
cd <CRIMINAL_EYE>\ai-service
.\.venv\Scripts\Activate.ps1
uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

Terminal 3 — Next.js

```
cd <CRIMINAL_EYE>
pnpm dev
```

Runtime path

```
Browser :3000
   ↓
Next.js server route
   ↓
FastAPI :8000
   ├── Qwen :8001
   └── Geometry → ControlNet → SD1.5
```

11.1 First end-to-end test

```
Witness:
"The person had a long oval face, narrow almond-shaped eyes
with normal spacing, thick arched eyebrows, a long straight
narrow nose with a rounded tip, thin upper lips, medium lower
lips, a medium rounded jaw and a rounded chin."

Expected:
1. Qwen returns valid JSON.
2. Pydantic validates it.
3. Next.js shows editable attributes.
4. Geometry creates the structural guide.
5. SD1.5 + ControlNet generates the sketch.
6. Seed/model metadata is stored.
```

12. RTX 4050 6 GB Performance Rules

Component	Start with	Avoid initially
Qwen	Q4_K_M GGUF	BF16/FP16 GPU load
SD1.5	FP16, 512×512	1024×1024
ControlNet	One Lineart model	Multiple ControlNets
Steps	20–30	60–100
Batch	1	>1
Concurrency	1 GPU generation	Parallel generations
Scheduling	Qwen and diffusion sequential	Both resident on GPU

12.1 If CUDA OOM occurs

- Confirm 512×512 and batch size 1.
- Enable model CPU offload and attention slicing.
- Do not run Qwen and diffusion heavily on the GPU at the same time.
- Reduce resolution before adding more models.
- Restart the service after repeated experiments if memory fragmentation appears.
- Measure peak VRAM and latency before each optimization.

Storage warning
Your screenshot shows about 290 GB used of 477 GB. Model caches, datasets and outputs can grow quickly. Keep one canonical copy of each model and monitor Hugging Face cache usage.

13. Testing & Phase 7 Upgrade Path

Test one layer at a time: Qwen → schema → geometry → SD1.5 → ControlNet → FastAPI → Next.js.

Test	Pass condition
NVIDIA	nvidia-smi sees RTX 4050
PyTorch	torch.cuda.is_available() is True
Qwen	localhost:8001 responds
Schema	Unsupported values are rejected
MediaPipe	Landmarks are returned
SD1.5	512×512 output works
ControlNet	Structural guide affects output
FastAPI	health + readiness work
Next.js	Server route reaches FastAPI
End-to-end	Witness → attributes → sketch works

13.1 Phase 7 training order

- A — Stable Qwen attribute extraction.
- B — Stable deterministic geometry.
- C — Stable SD1.5 + ControlNet baseline.
- D — LoRA adaptation to the target sketch style.
- E — Component-aware eyes/nose/mouth conditioning.
- F — Quantitative and human evaluation.

Evaluation targets

- Attribute fidelity: intended features are preserved.
- Geometric stability: changing one feature does not randomly change unrelated regions.
- Style fidelity: output matches the target sketch style.
- Reproducibility: seed and model versions are stored.
- Human review: evaluators judge the result.

Training reality
Your 6 GB GPU is appropriate for inference and small experiments, not large research-model training. Use larger compute only when a training experiment genuinely requires it.

14. Security, Licensing & Final Checklist

Treat witness descriptions, images and potential biometric data as sensitive application data.

Security

- Bind Qwen to 127.0.0.1 in local development.
- Never expose port 8001 to the browser.
- Authenticate FastAPI in production.
- Validate image MIME types, size and dimensions.
- Use generated storage names, not trusted user filenames.
- Avoid logging raw witness descriptions by default.
- Store model versions, schema version, seed and generation parameters.
- Use authenticated or signed access for generated files.

Final Definition of Done

01. ☐ RTX 4050 visible to the NVIDIA driver.
02. ☐ Python 3.11 virtual environment works.
03. ☐ CUDA-enabled PyTorch verified.
04. ☐ Qwen3-8B Q4_K_M runs through llama.cpp.
05. ☐ FastAPI health endpoint works.
06. ☐ Qwen output passes strict schema validation.
07. ☐ MediaPipe geometry layer works.
08. ☐ SD1.5 generates 512×512 output.
09. ☐ ControlNet Lineart follows the structural guide.
10. ☐ Next.js proxies requests server-side.
11. ☐ User can review/edit attributes before generation.
12. ☐ Sketch metadata stores seed and model versions.
13. ☐ FS2K is acquired from official repository instructions.
14. ☐ LoRA training begins only after baseline stability.

Responsible-use boundary
A generated sketch is a reconstruction aid, not proof of identity. Do not use a generated image or similarity score as an autonomous determination of guilt or identity. Keep qualified human review in the workflow.

Official references

Qwen3-8B GGUF
https://huggingface.co/Qwen/Qwen3-8B-GGUF

Qwen3-8B Q4_K_M
https://huggingface.co/Qwen/Qwen3-8B-GGUF/blob/main/Qwen3-8B-Q4_K_M.gguf

llama.cpp releases
https://github.com/ggml-org/llama.cpp/releases

Stable Diffusion 1.5
https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5

ControlNet Lineart
https://huggingface.co/lllyasviel/control_v11p_sd15_lineart

FS2K
https://github.com/DengPingFan/FS2K

MediaPipe Face Landmarker
https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker/python

Diffusers
https://huggingface.co/docs/diffusers/installation

PyTorch
https://pytorch.org/get-started/locally/

FastAPI
https://fastapi.tiangolo.com/tutorial/first-steps/